TEMPO: An ML-KEM to PAKE compiler resilient to timing attacks

Afonso Arriaga[1], Manuel Barbosa[2], Stanislaw Jarecki[3]

(1) SnT - University of Luxembourg, Esch-sur-Alzette, Luxembourg
(2) FCUP, University of Porto, Porto, Portugal
(3) University of California, Irvine, USA

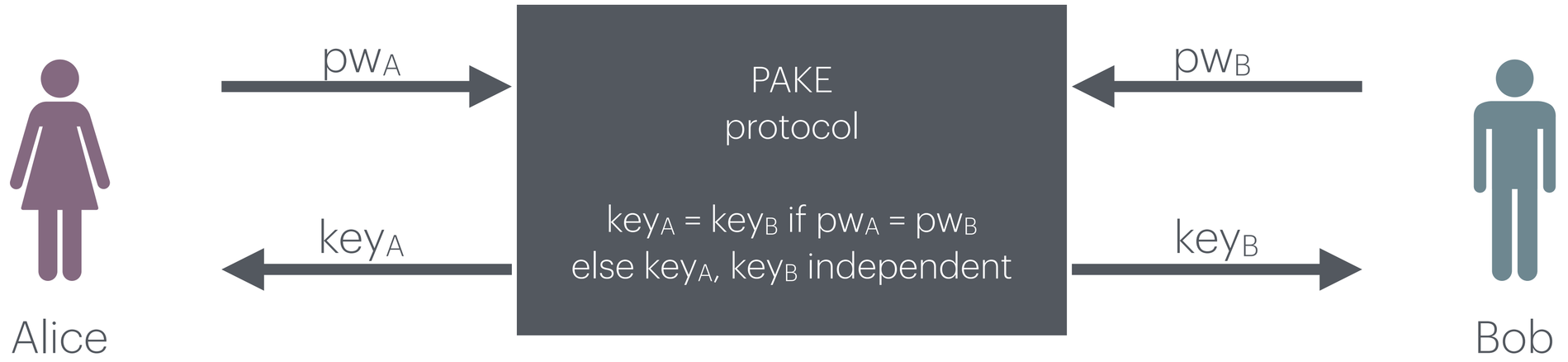
CHES'26, Antalya, Turkey

UNIVERSITÉ DU LUXEMBOURG

SnT

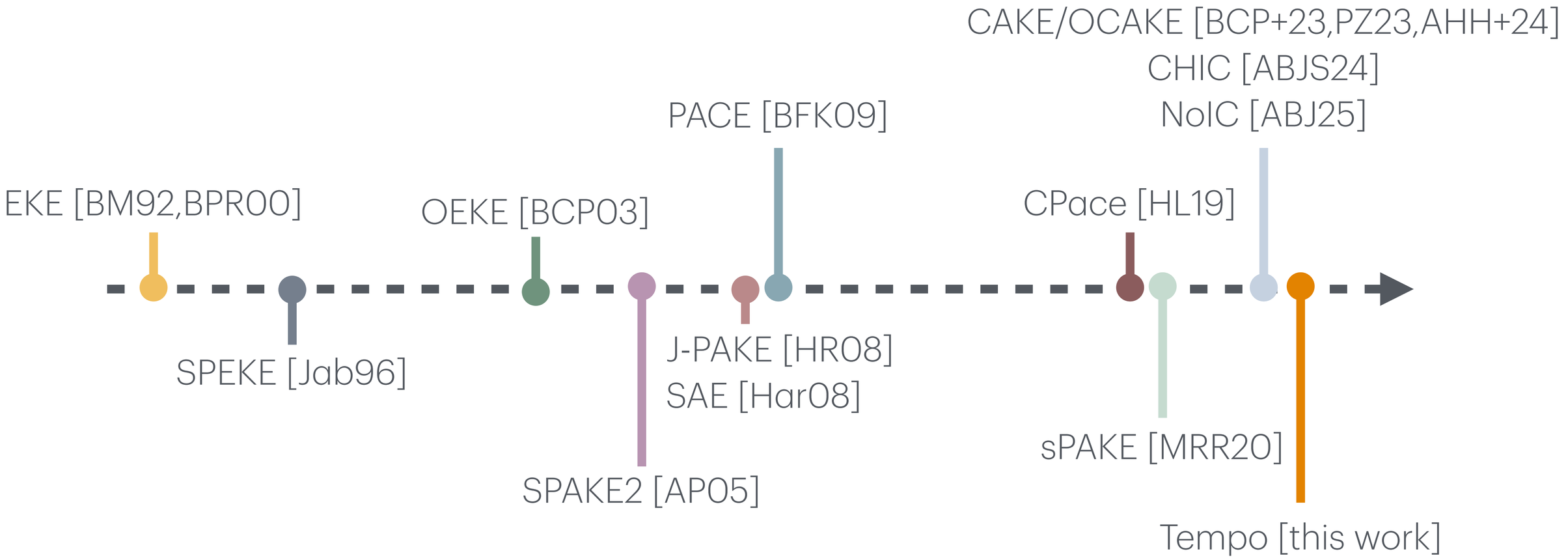
PAKE: Password Authenticated Key Exchange

Protocol



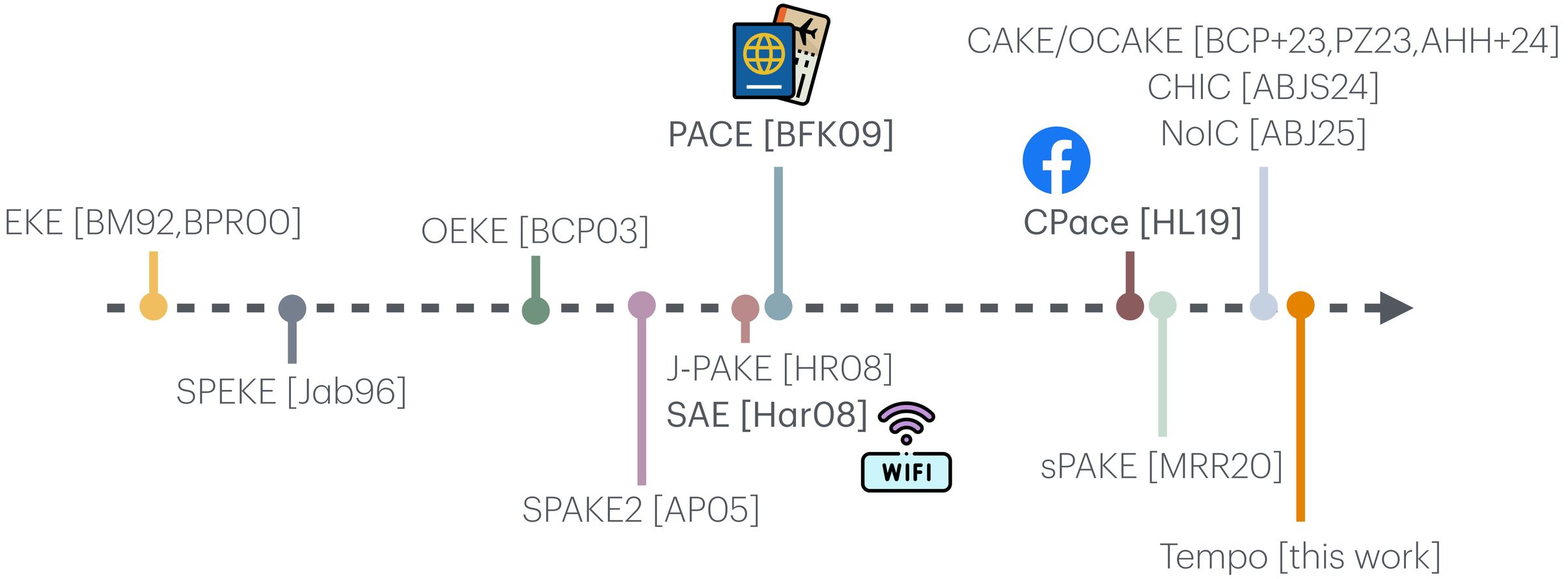
PAKE: Password Authenticated Key Exchange

Timeline



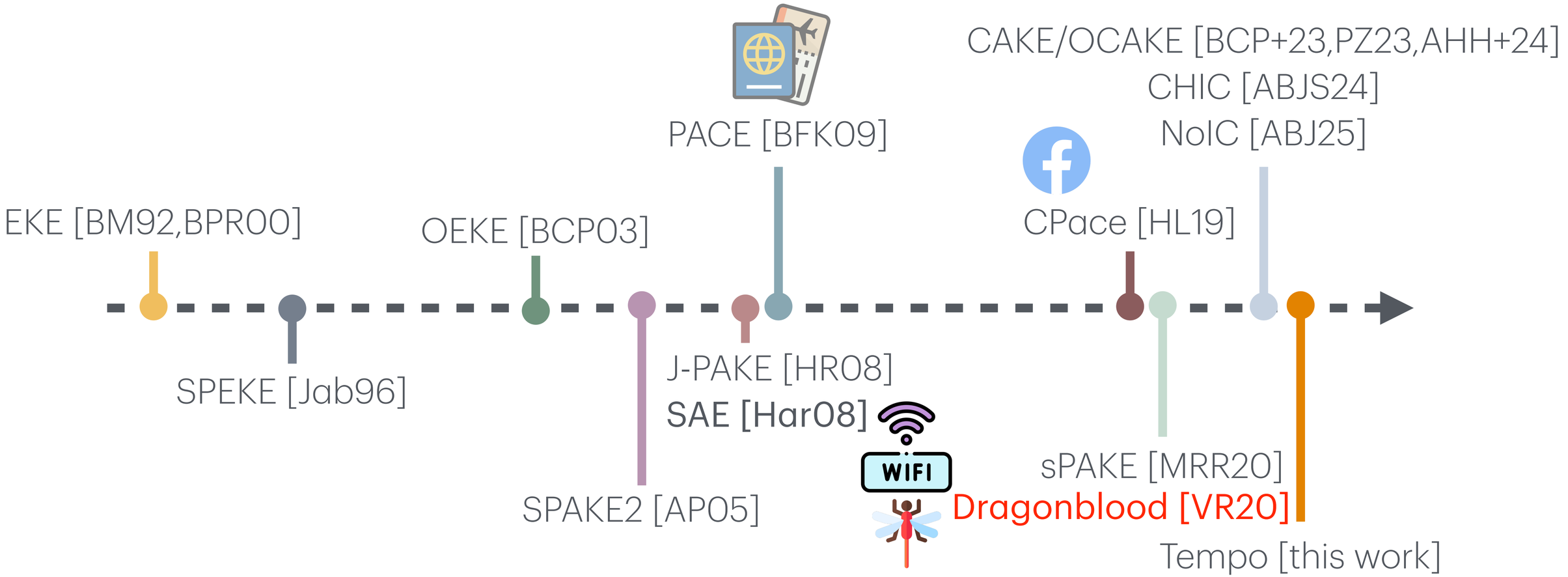
PAKE: Password Authenticated Key Exchange

Real-world adoption of symmetric PAKE protocols



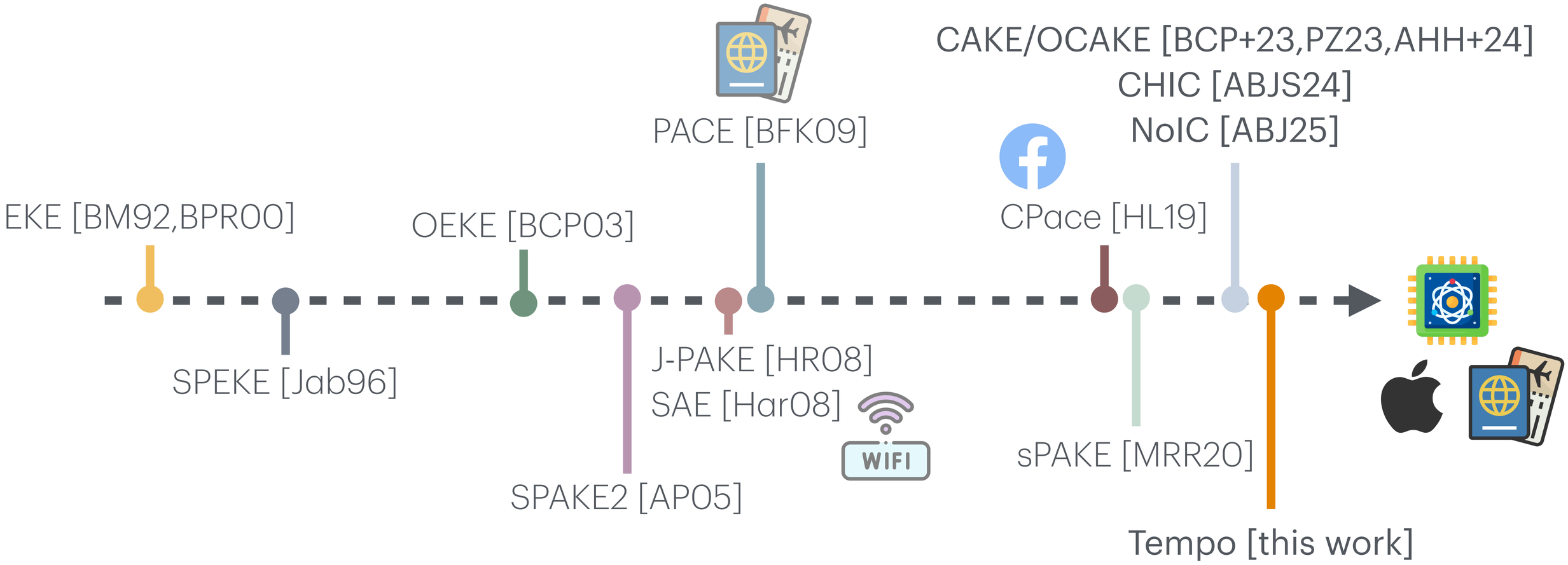
PAKE: Password Authenticated Key Exchange

Real-world attacks



PAKE: Password Authenticated Key Exchange

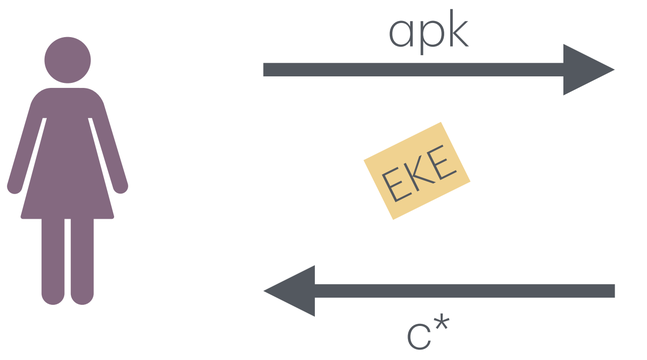
Towards post-quantum security...



EKE and OEKE design patterns

(sk, pk) ← KEM.Keygen
apk ← **IC.Enc(pw_A, pk)**

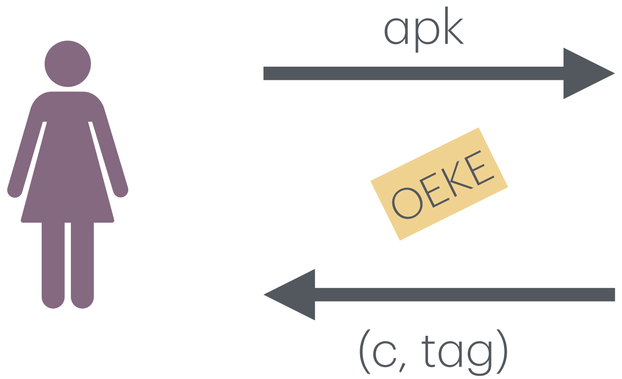
c ← IC.Dec(pw_A, c*)
K ← KEM.Decaps(sk, c)
key ← H(pk, apk, c, K, ...)



pk ← IC.Dec(pw_B, apk)
(K, c) ← KEM.Encaps(pk)
c* ← **IC.Enc(pw_B, c)**
key ← H(pk, apk, c, K, ...)

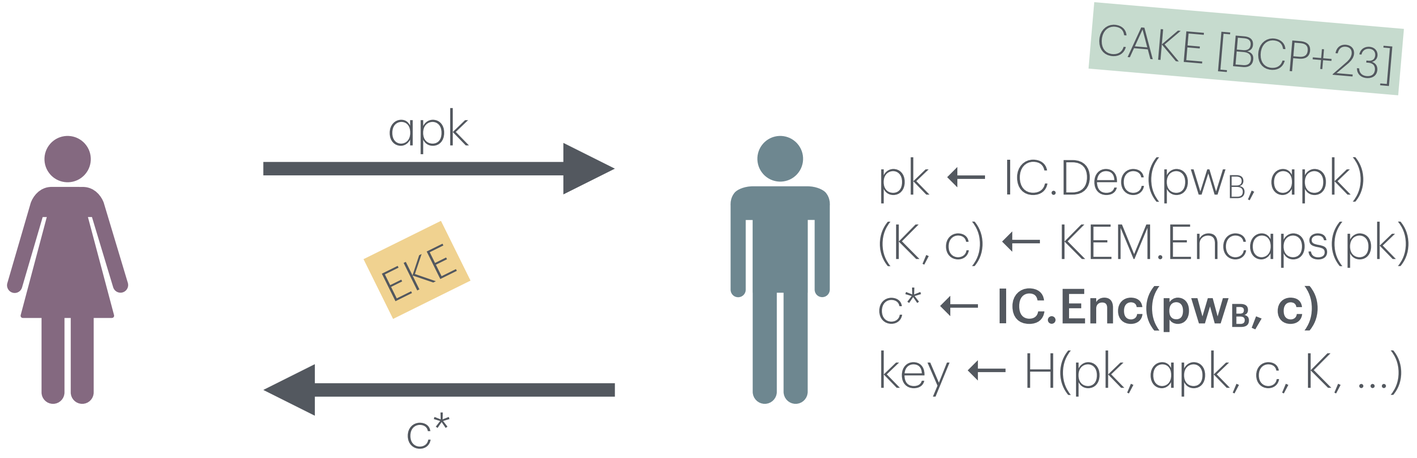
(sk,pk) ← KEM.Keygen
apk ← **IC.Enc(pw_A, pk)**

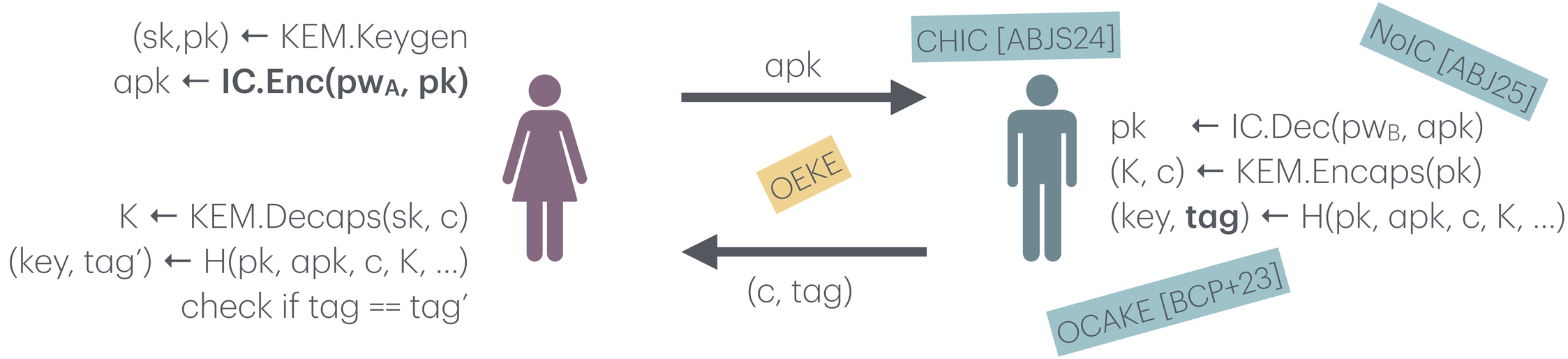
K ← KEM.Decaps(sk, c)
(key, tag') ← H(pk, apk, c, K, ...)
check if tag == tag'



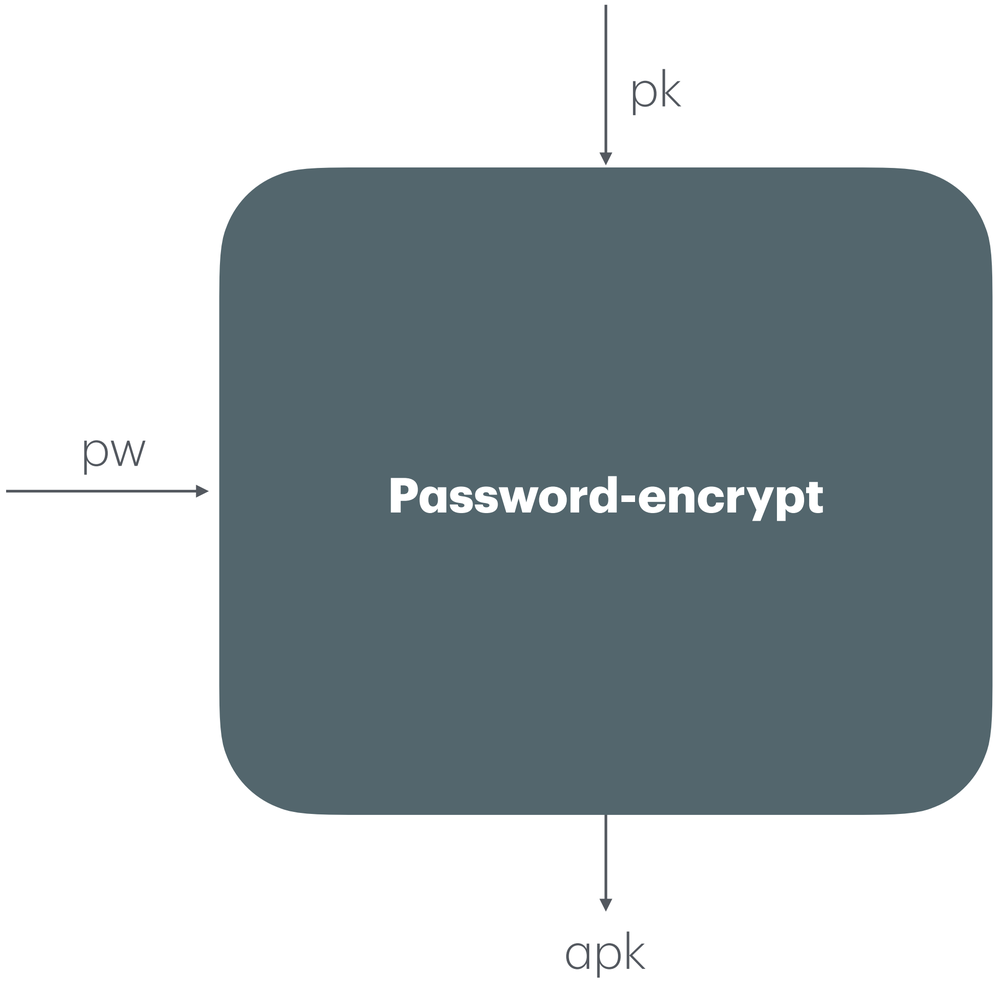
pk ← IC.Dec(pw_B, apk)
(K, c) ← KEM.Encaps(pk)
(key, **tag**) ← H(pk, apk, c, K, ...)

EKE and OEKE design patterns





Problem #1: how to instantiate the IC?



Problem #1: how to instantiate the IC?

- Usually, ideal ciphers are instantiated with block ciphers.
- PQ KEMs public keys and the ciphertexts do not fit into a single block!
- The domain of the IC and its instantiation must coincide; otherwise, it becomes trivial for an adversary to mount an offline dictionary attack. The attack proceeds as follows:

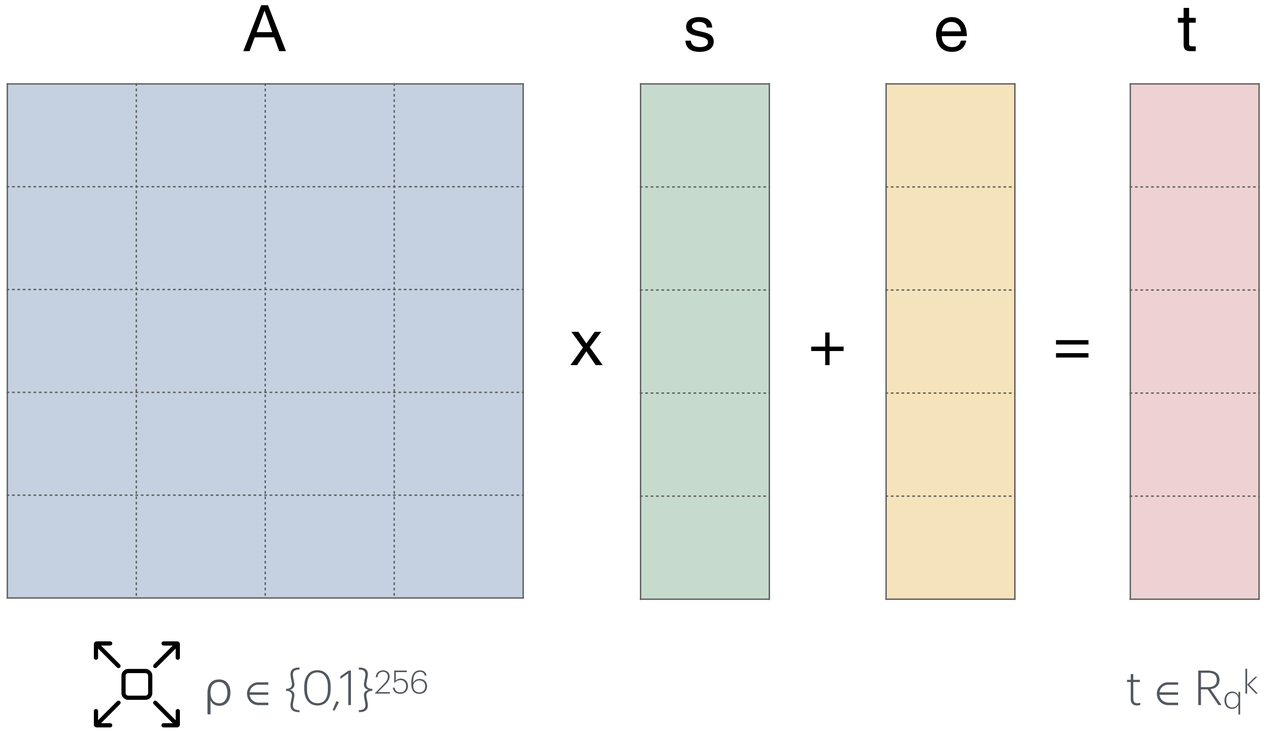
    1. Intercept apk = IC.Enc(pw, pk).

    2. Decrypt apk using a candidate password pw*.

    3. If pk* = IC.Dec(pw*, apk) does not belong to the public key space, then pw* is incorrect.

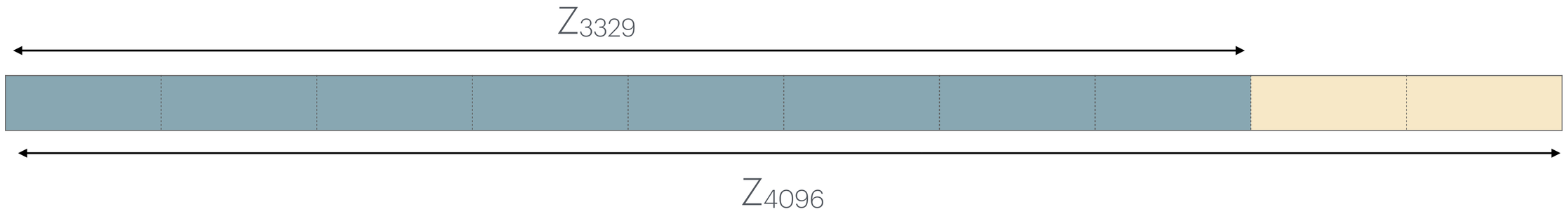
ML-KEM public keys (FIPS 203)

- Sample $\rho \in \{0,1\}^{256}$
- $A \leftarrow \text{Expand}(\rho) \in R_q^{k*k}$
- $t \leftarrow A * s + e$
- $pk \leftarrow (\rho, t)$

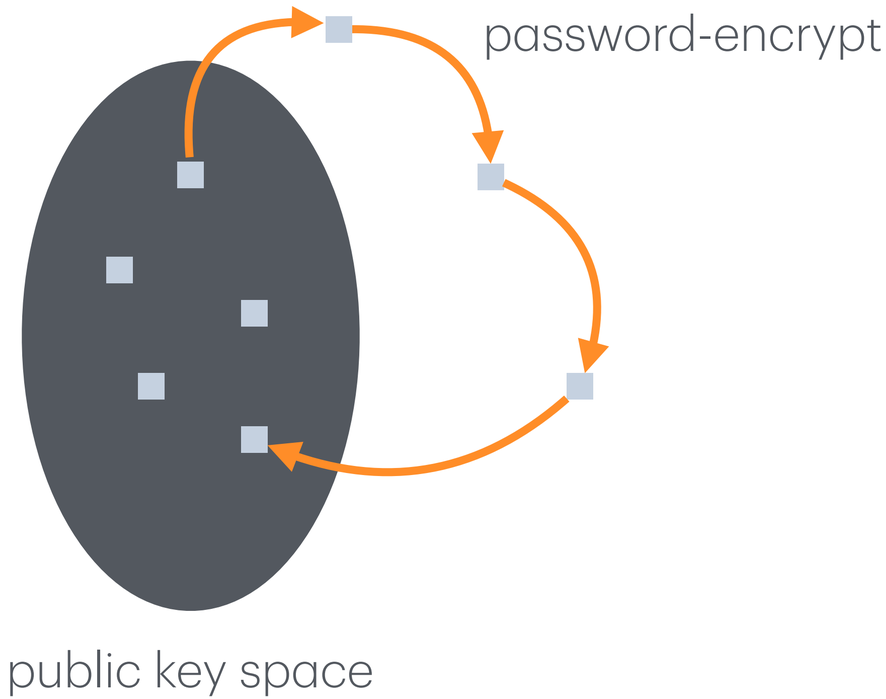


ML-KEM public keys

- A is a k*k matrix of polynomials with 256 coefficients each, modulo q.
- FIPS 203 specifications set q = 3329
- $2^{11} < 3329 < 2^{12}$
- To encode a single element in Z_{3329}, we need 12 bits



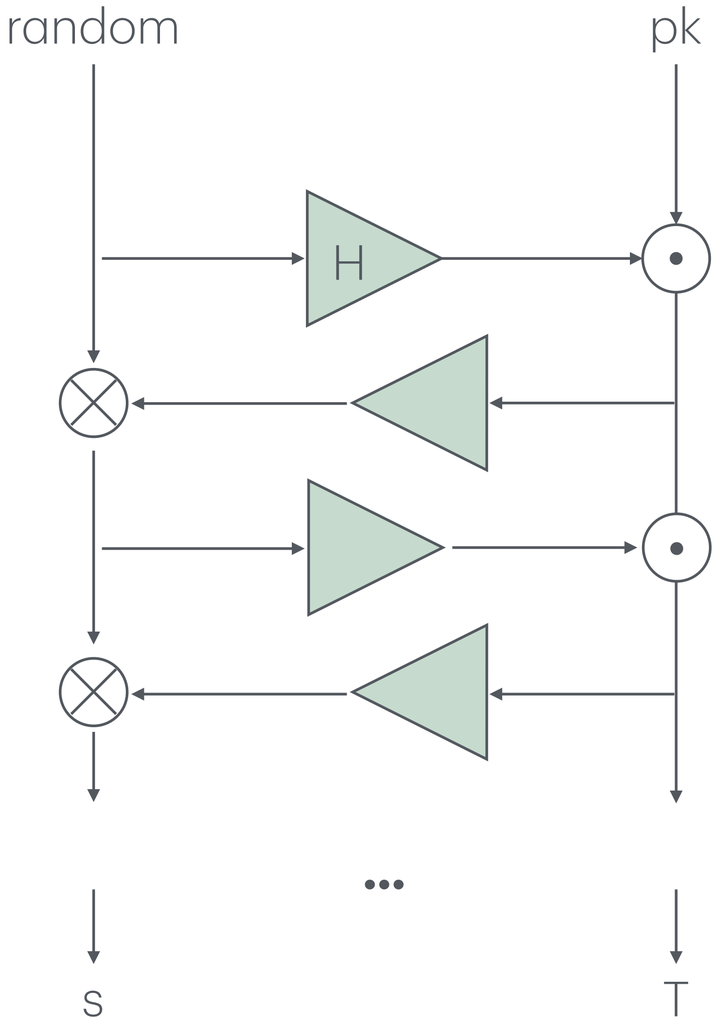
Candidate solutions



1. We can use domain extenders to enlarge the input/output domain of a block cipher so that it covers the public-key space of the KEM.
2. Then use the cycle walking method [BCP+23].

cycle walking method

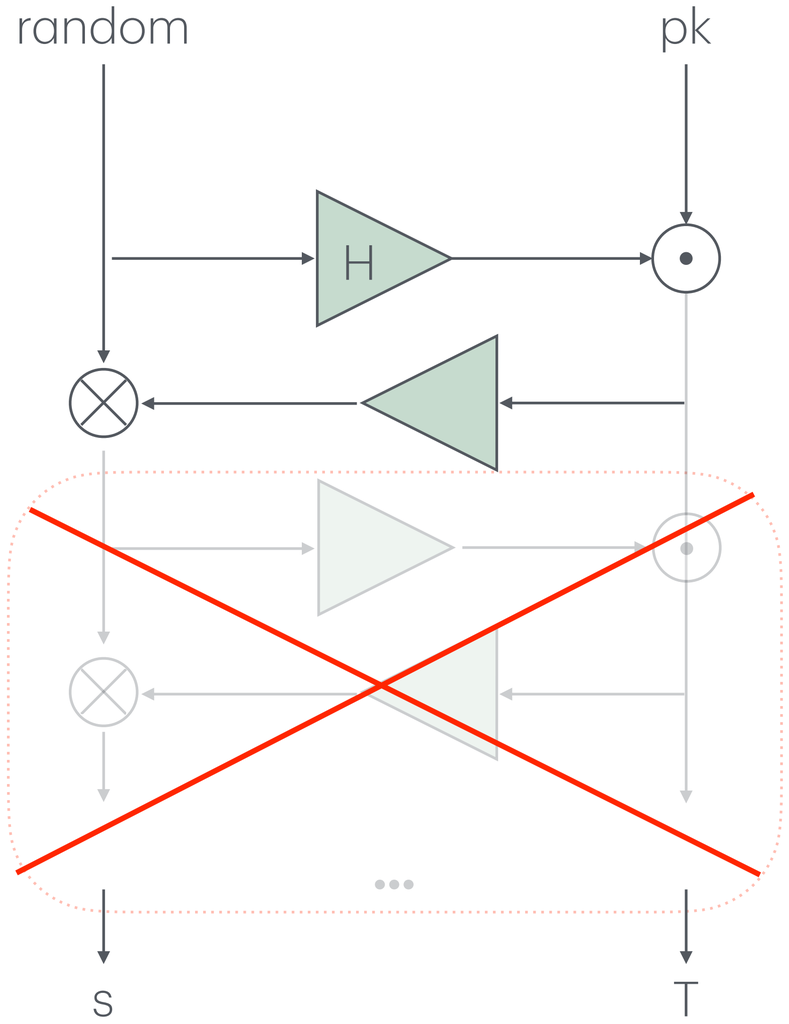
Candidate solutions



1. We can use an 8-round Feistel network [DS16], placing the public key on one wire and random coins on the other.
2. We can instantiate the hash-to-group function H with rejection sampling [ABJS24].

8-round Feistel is indifferentiable from IC

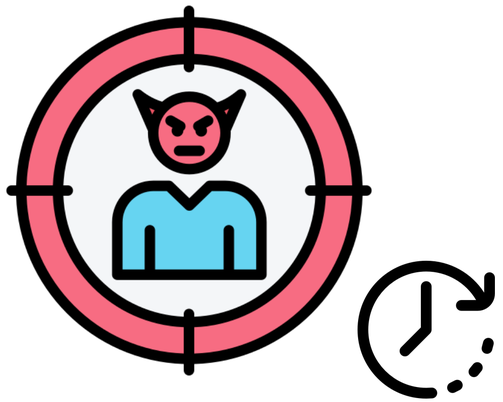
Candidate solutions



KEM-to-PAKE compilers only need 2 rounds [JRX25, ABJ25]

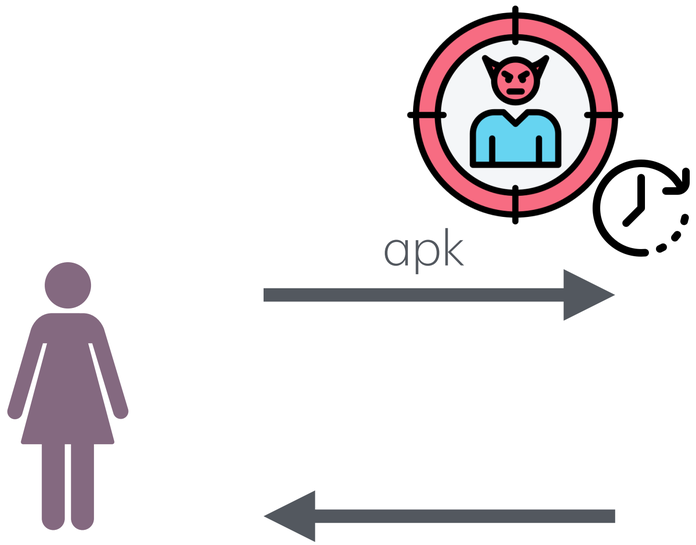
Problem #2: timing attacks

- Password dictionary is relatively small
- Cycle walking and rejection sampling methods have variable-time dependent on the password!



The attack

(sk,pk) ← KEM.Keygen
apk ← IC.Enc(pw_A, pk)



pk ← IC.Dec(pw_B, apk)
(K, c) ← KEM.Encaps(pk)
(key, tag) ← H(pk, apk, c, K, …)

K ← KEM.Decaps(sk, c)
(key, tag') ← H(pk, apk, c, K, …)
check if tag == tag'

Find which passwords are consistent with time measurements on Alice or Bob

The attack

The problem compounds with ML-KEM pk expansion step, which also uses rejection sampling !!!



→ (sk,pk) ← KEM.Keygen
apk ← IC.Enc(pw_A, pk)

apk

pk ← IC.Dec(pw_B, apk)
(K, c) ← KEM.Encaps(pk) ←
(key, tag) ← H(pk, apk, c, K, …)

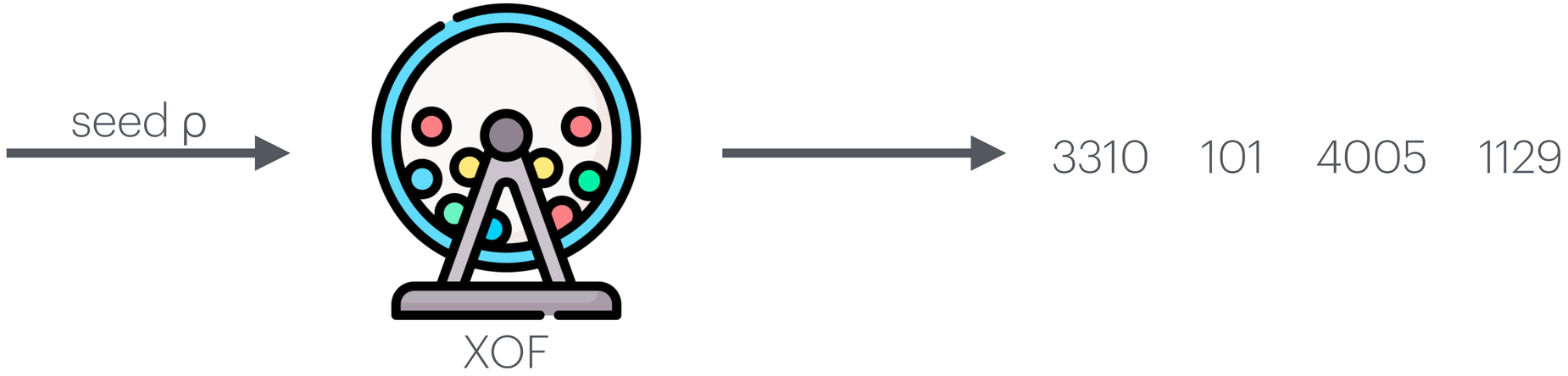
→ K ← KEM.Decaps(sk, c)
(key, tag') ← H(pk, apk, c, K, …)
check if tag == tag'

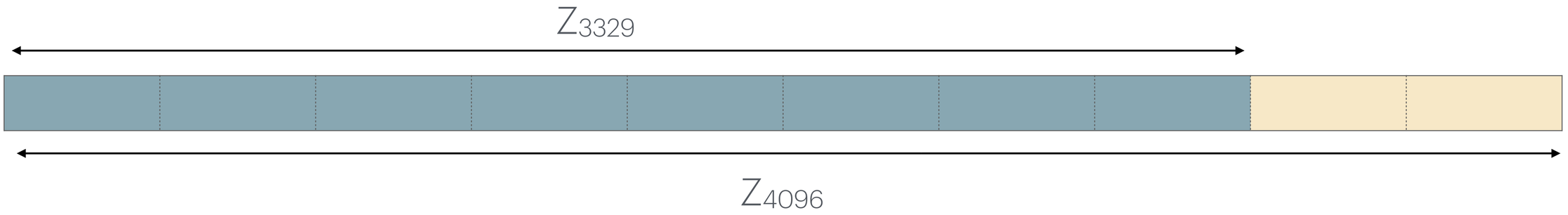
(c, tag)

Find which passwords are consistent with time measurements on Alice or Bob

Solution #1: Make sampling methods constant-time

1. Fixed Loop Sampler (FLS):
   - Oversample a fixed number of samples
   - May fail with statistically negligible probability
   - Compatible with FIPS 203

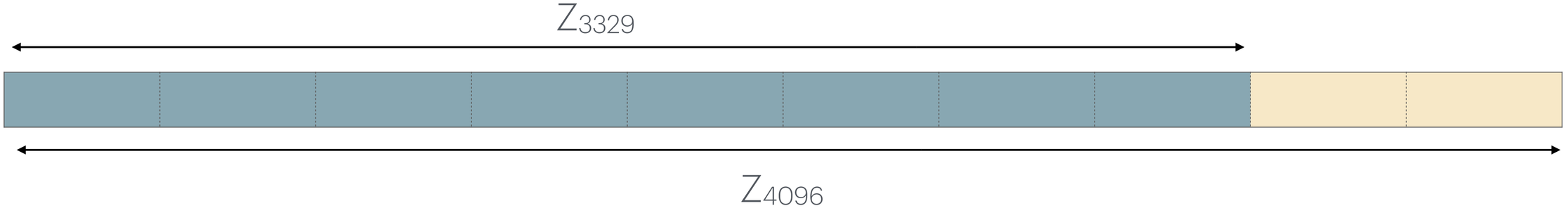




Solution #1: Make sampling methods constant-time

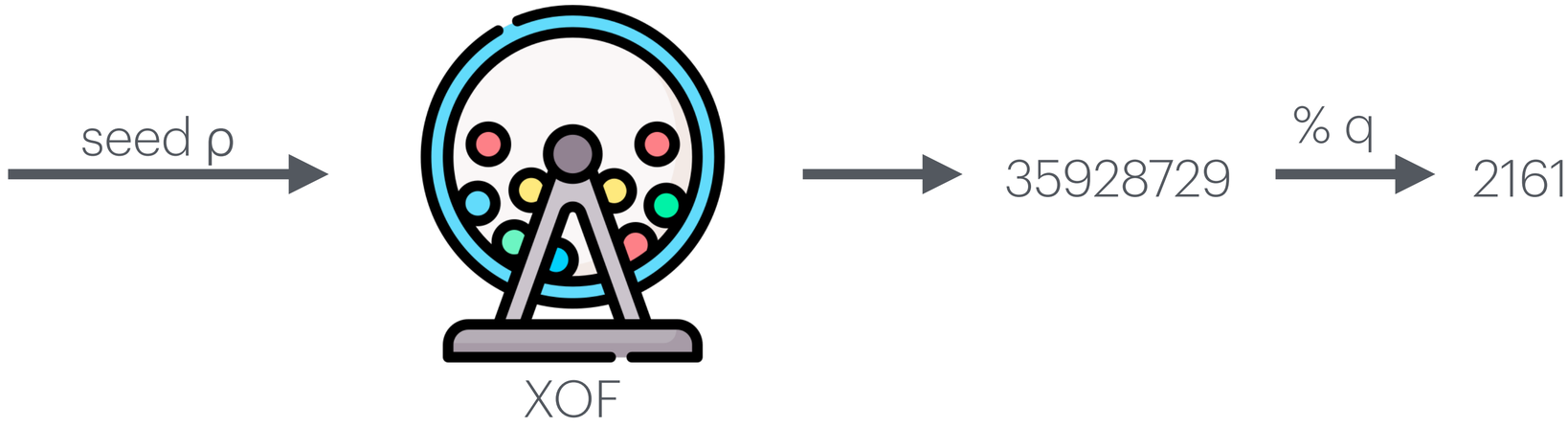
2. Repeated modular division (i.e., the “Kemeleon” method [GSV24]):
   - Expand seed to a very large integer (large enough to extract all necessary coefficients)
   - Repeatedly divide by q=3329 to extract coefficients

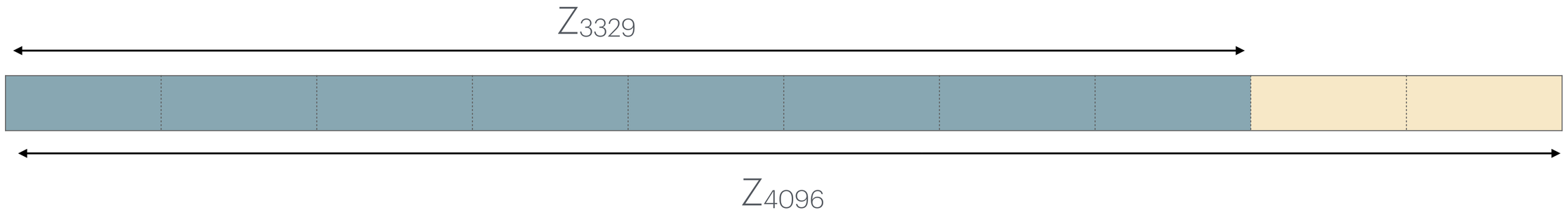




Solution #1: Make sampling methods constant-time

3. Single modular devision
   - Expand to 192-bit integers and extract 1 coefficient at the time





Solution #1: Make sampling methods constant-time

Benchmarks on an x86 laptop

	Implementations of SampleNTT			
	Algorithm FIPS 203	Algorithm FLS	Algorithm RMD	Algorithm SMD
	standalone, variable-time	standalone	OpenSSL	standalone
Avg time to generate matrix A $\forall i, j \in [0, k),\ A[i, j] \leftarrow\$ \mathsf{SampleNTT}(\rho \| i \| j)$, where $k \in \{2, 3, 4\}$ is determined by the ML-KEM security parameter λ				
MLKEM-512.gen_a	68 974±2.12%	487 938±0.13%	1 174 719±0.93%	484 339±0.04%
MLKEM-768.gen_a	136 485±1.09%	1 080 660±0.09%	2 621 815±0.50%	1 072 411±0.23%
MLKEM-1024.gen_a	232 905±0.82%	1 906 313±0.15%	4 649 635±0.34%	1 895 492±0.05%
Avg time to execute MLKEM (keygen+enc+dec) SampleNTT used to (re-)generate matrix A				
MLKEM-512	712 717±0.38%	1 972 312±0.05%	4 030 149±0.30%	1 959 798±0.05%
MLKEM-768	1 213 602±0.41%	4 052 170±0.08%	8 622 886±0.21%	4 014 919±0.07%
MLKEM-1024	1 803 419±0.33%	6 835 985±0.06%	15 064 128±0.53%	6 796 320±0.06%
Avg time to execute NoIC PAKE (start+resp+end) SampleNTT used both to (re-)generate matrix A and instantiate H_1				
NoIC[MLKEM-512]	973 654±0.29%	2 650 184±0.12%	5 392 876±0.30%	2 634 712±0.13%
NoIC[MLKEM-768]	1 575 863±0.35%	5 033 625±0.06%	10 647 686±0.14%	5 011 368±0.06%
NoIC[MLKEM-1024]	2 295 495±0.29%	8 162 881±0.06%	17 752 864±0.17%	8 117 978±0.06%

Variable-time rejection sampling is the fastest

Constant-time methods are slower

Solution #1: Make sampling methods constant-time

Benchmarks on an x86 laptop

	Implementations of SampleNTT			
	Algorithm FIPS 203	Algorithm FLS	Algorithm RMD	Algorithm SMD
	standalone, variable-time	standalone	OpenSSL	standalone
Avg time to generate matrix A $\forall i,j \in [0,k),\ A[i,j] \leftarrow\$ \mathsf{SampleNTT}(\rho\|i\|j)$, where $k \in \{2,3,4\}$ is determined by the ML-KEM security parameter λ				
MLKEM-512.gen_a	68 974±2.12%	487 938±0.13%	1 174 719±0.93%	484 339±0.04%
MLKEM-768.gen_a	136 485±1.09%	1 080 660±0.09%	2 621 815±0.50%	1 072 411±0.23%
MLKEM-1024.gen_a	232 905±0.82%	1 906 313±0.15%	4 649 635±0.34%	1 895 492±0.05%
Avg time to execute MLKEM (keygen+enc+dec) SampleNTT used to (re-)generate matrix A				
MLKEM-512	712 717±0.38%	1 972 312±0.05%	4 030 149±0.30%	1 959 798±0.05%
MLKEM-768	1 213 602±0.41%	4 052 170±0.08%	8 622 886±0.21%	4 014 919±0.07%
MLKEM-1024	1 803 419±0.33%	6 835 985±0.06%	15 064 128±0.53%	6 796 320±0.06%
Avg time to execute NoIC PAKE (start+resp+end) SampleNTT used both to (re-)generate matrix A and instantiate H_1				
NoIC[MLKEM-512]	973 654±0.29%	2 650 184±0.12%	5 392 876±0.30%	2 634 712±0.13%
NoIC[MLKEM-768]	1 575 863±0.35%	5 033 625±0.06%	10 647 686±0.14%	5 011 368±0.06%
NoIC[MLKEM-1024]	2 295 495±0.29%	8 162 881±0.06%	17 752 864±0.17%	8 117 978±0.06%

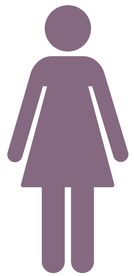
~3.3x slower

Variable-time rejection sampling is the fastest

Constant-time methods are slower

Solution #2: Tempo protocol

(sk, pk) ← KEM.Keygen
(t, ρ) ← KEM.Split(pk)
apk ← 2F.Enc(pw_A, t)



(apk, ρ)





t ← 2F.Dec(pw_B, apk)
pk ← (t, ρ)
(K, c) ← KEM.Encaps(pk)
(key, tag) ← H(pk, apk, c, K, ...)



(c, tag)

K ← KEM.Decaps(sk, c)
(key, tag') ← H(pk, apk, c, K, ...)
check if tag == tag'

Solution #2: Tempo protocol

(sk, pk) ← KEM.Keygen
(t, ρ) ← KEM.Split(pk)
apk ← 2F.Enc(pw_A, t)

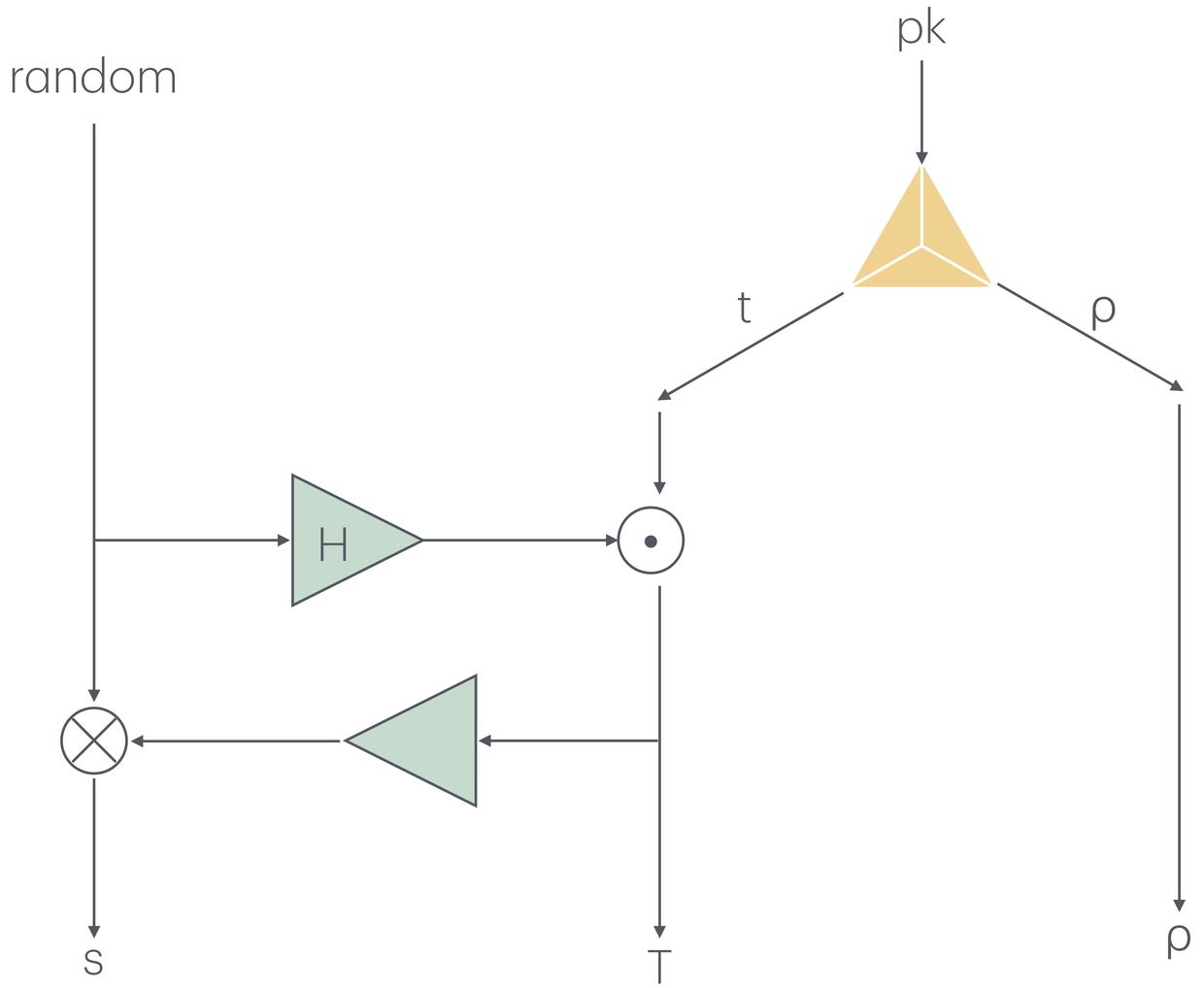


t ← 2F.Dec(pw_B, apk)
pk ← (t, ρ)
(K, c) ← KEM.Encaps(pk)
(key, tag) ← H(pk, apk, c, K, …)

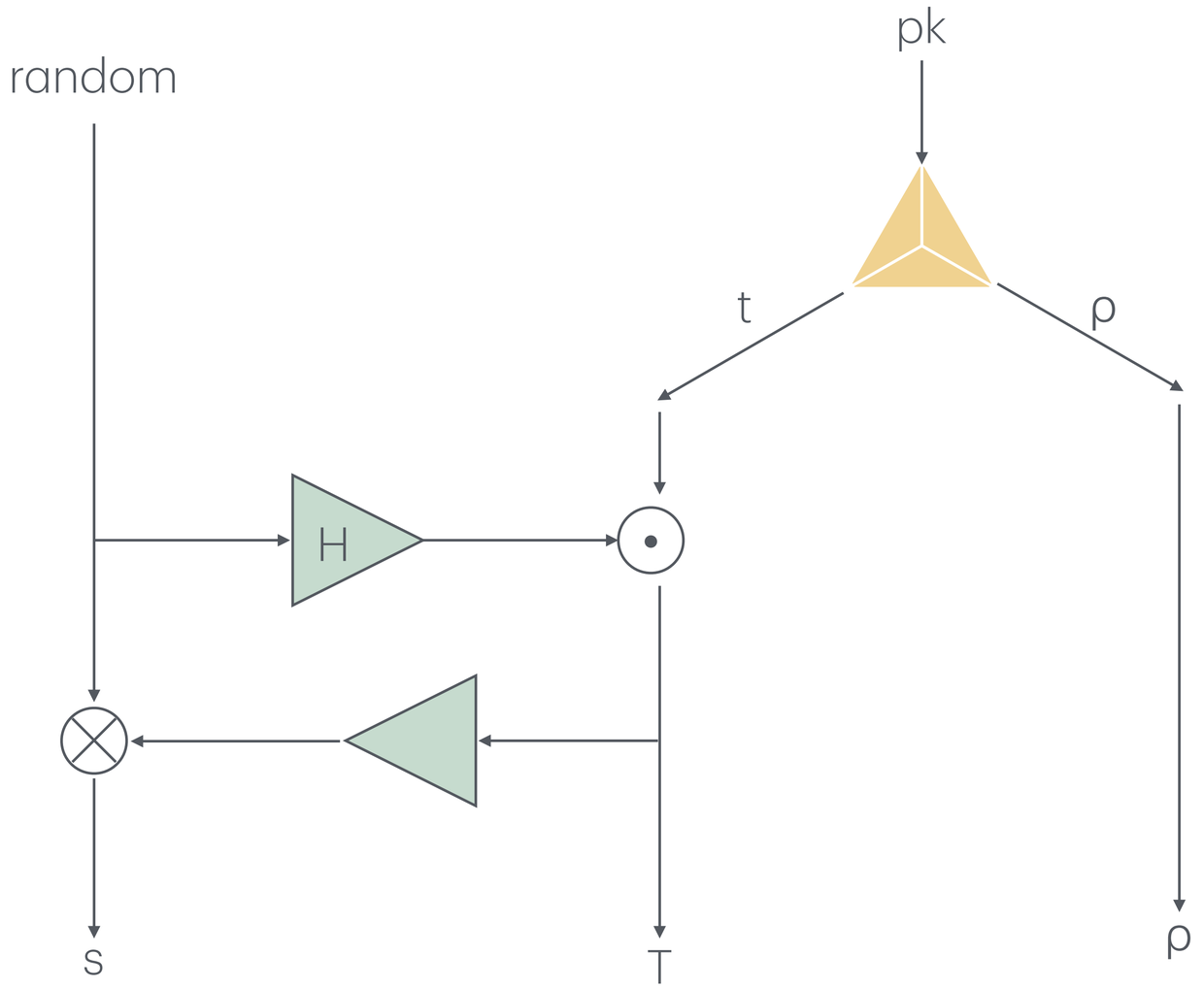
K ← KEM.Decaps(sk, c)
(key, tag') ← H(pk, apk, c, K, …)
check if tag == tag'

We still need to instantiate the hash-to-group with constant-time methods, but ML-KEM can be used unchanged

Solution #2: Tempo protocol

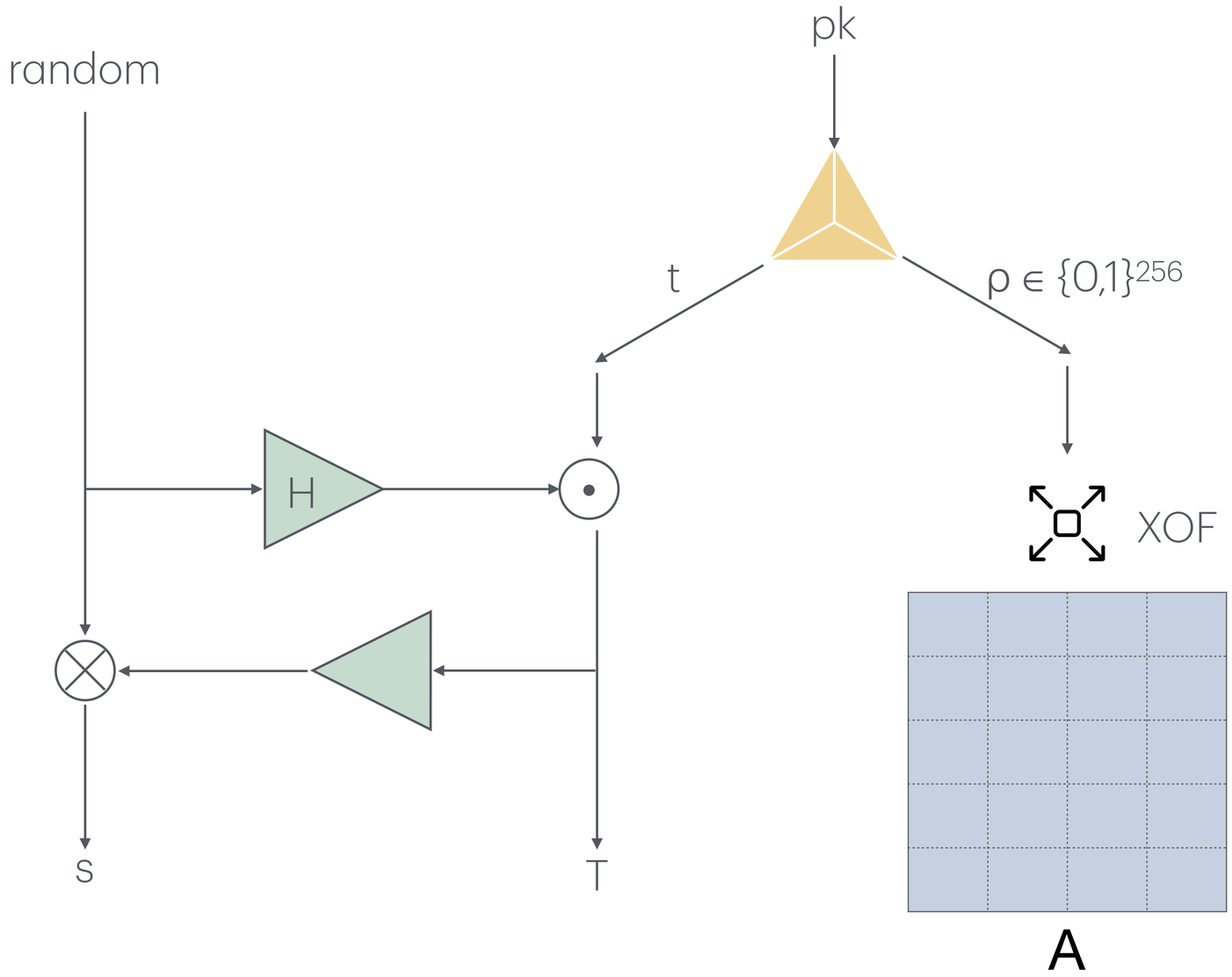


Solution #2: Tempo protocol



caveat: ρ is adversarially chosen

Solution #2: Tempo protocol



Oracle-MLWE is asymptotically equivalent to MLWE when XOF is modeled as an RO

caveat: ρ is adversarially chosen

Solution #2: Tempo protocol

Avg cycles to execute Tempo PAKE Algorithm SMD used only to instantiate H_1 ML-KEM reference implementation without modifications			
	start	response	end
Tempo[ML-KEM-512]	1 605 498±1917	2 064 550±1915	1 247 926±1915
Tempo[ML-KEM-768]	2 546 709±3220	3 173 110±3226	1 937 037±3220
Tempo[ML-KEM-1024]	3 612 553±4620	4 416 318±4622	2 740 962±4621

Avg cycles to execute ML-KEM reference implementation			
	keygen	encaps	decaps
ML-KEM-512	596 338±2209	701 146±2210	889 195±2210
ML-KEM-768	987 580±3244	1 137 080±3244	1 386 842±3245
ML-KEM-1024	1 536 967±2874	1 708 715±2878	2 020 960±2879

~ 2.2x

Benchmarks on
STM32 Nucleo-144 board

Conclusions

- ML-KEM public keys are meant to be public, so the ρ → A expansion can be variable-time.
- But most KEM-to-PAKE compilers (e.g., CAKE, OCAKE, CHIC, NoIC) encrypt the KEM public key under a secret password. A variable-time expansion then leaks information about the password.
- Instantiating the password-encryption layer is also problematic if it relies on variable-time methods (e.g., cycle walking, rejection sampling).
- Tempo reveals ρ in the clear, so ML-KEM runs unchanged.
- Tempo costs only about 2x the underlying ML-KEM.

Thank you for your attention